

Université de Montréal

Application de la programmation en nombres entiers aux
cryptosystèmes post-quantiques basés sur les réseaux

Alexis Lessnick
20297698

Département d'informatique et de recherche opérationnelle
Faculté des arts et des sciences

Travail supervisé par Prof. Kim Yu
dans le cadre du cours IFT3150

Août 2026

Résumé

Dans ce travail, nous analysons les liens pouvant être tissés entre la programmation en nombres entiers et les problèmes de cryptographie post-quantique basés sur les réseaux (*lattices*). Nous étudions principalement le problème *Learning With Errors* (LWE), celui-ci ayant été désigné par le *NIST* comme étant le prochain standard de chiffrement pour les années à venir. Nous implémentons alors une attaque sur LWE via le solveur Gurobi : celle-ci se base sur la réduction en programmation en nombres entiers du problème Search-LWE évoquée dans une prépublication de M. Shirase [Cryptology ePrint Archive (2023)]. En quantifiant les performances numériques des tentatives de déchiffrement utilisant cette modélisation, il est possible de constater que celles-ci ne sont pas très efficaces lorsque comparées aux attaques de l'état de l'art. Cela justifie alors la nécessité d'apporter des retouches à notre modèle initial. Nous proposons donc une construction algébrique alternative permettant l'application d'un prétraitement par l'algorithme de réduction de réseaux LLL, ce qui a pour effet d'accélérer significativement le temps de résolution de petites instances de LWE : nous observons une réduction du temps de calcul de l'ordre de 10^4 secondes.

Notation

$\mathbf{v}$: Un vecteur

v_i : $i^{\text{ème}}$ composante d'un vecteur $\mathbf{v}$

$\|v\|$: La norme euclidienne d'un vecteur $\mathbf{v}$

q : Un nombre premier

$\mathbb{R}^n$: Espace vectoriel de dimension n sur $\mathbb{R}$

$\mathbb{Z}^n(\subset \mathbb{R}^n)$: Sous-ensemble des composantes entières de $\mathbb{R}^n$

$\mathbb{Z}_q$: $\{0, 1, 2, \dots, q-1\}$

$\mathbb{Z}_q^n$: Espace vectoriel de dimension n sur $\mathbb{Z}_q$

$\mathbb{Z}_q^{m \times n}$: Ensemble des matrices de dimensions $n \times m$ aux composantes dans $\mathbb{Z}_q$

$[-X, X]^n$: Ensemble des vecteurs de $\mathbb{Z}^n$ dont chaque composante est dans $[-X, X]$

$e \in_R A$: Le vecteur e est choisi aléatoirement de manière uniforme depuis A

$\lfloor x \rfloor$: Arrondi de x à l'entier le plus proche

Table des matières

Résumé	i
Notation	ii
Introduction	1
1 Notions préliminaires	3
2 État de l’art	6
3 Méthodologie	9
4 Modélisation initiale	11
4.1 Cadre mathématique et implémentation	11
4.2 Difficultés rencontrées	14
4.3 Résultats préliminaires	15
5 Cryptosystèmes actuels et réduction $\text{MLWE} \leq \text{LWE}$	19
6 Améliorations	21
6.1 Modélisation CMOD	21
6.2 Modélisation LLL (erronée)	24
6.3 Modélisation LLL (adéquate)	26
Conclusion	30
Références	32
Remerciements	34
Annexe	35

Introduction

En 1994, Shor [14] publie un article révolutionnaire. Dans celui-ci, il argumente, via un algorithme portant désormais son nom, qu'un ordinateur quantique peut rapidement factoriser un nombre entier. Advenant alors la conception d'une telle machine, une quantité importante d'implémentations cryptographiques actuelles, telle que celle du cryptosystème RSA, ne seraient plus viables. En effet, la sécurité de RSA dépend directement de la difficulté qu'a un ordinateur classique à résoudre un tel problème de factorisation : l'ordinateur quantique le rendrait alors trivial [15]. Pour faire face à cette éventualité, il fut impératif de proposer des systèmes de cryptographie alternatifs, que l'on suppose être résistants aux potentielles technologies quantiques. De cette motivation naquit le domaine de la cryptographie post-quantique, définie naturellement comme étant l'ensemble des techniques de chiffrement classiques que l'on considère résistantes aux attaques quantiques.

Plusieurs cryptosystèmes furent alors théorisés par des mathématiciens du monde entier. Il fut donc possible de les regrouper en catégories. À ce jour, la famille de la cryptographie basée sur les réseaux est la mieux documentée. Parmi celle-ci existe le problème *Learning With Errors*, que l'on abrégera à LWE. Des implémentations qui lui sont analogues furent globalement standardisées dans les dernières années [8, 9] pour leur robustesse de chiffrement, tant contre un adversaire quantique qu'un adversaire classique. Cette standardisation découle de preuves théoriques ainsi que de l'échec d'attaques variées. Étonnamment, l'optimisation mathématique ne fut que très rarement considérée comme véhicule de déchiffrement. Dans les quelques travaux déjà entrepris [3, 7], le modèle d'optimisation découle souvent du monde quantique, ce qui le rend difficilement exécutable sans expertise poussée ni matériel spécialisé.

Notre proposition de projet se divise en deux temps. Dans une optique d'inclusivité, nous proposons pour commencer la rédaction d'une revue de littérature sur le sujet, laquelle se devant de décrire les problèmes cryptographiques à l'étude, d'analyser leur pertinence et leur origine, tout en présentant l'état de l'art quant à l'attaque de ceux-ci. Une telle mise en commun de ces savoirs servira de référence à quiconque

cherchant à s'initier au sujet : notons que nous visons ici principalement les étudiants au baccalauréat. En second lieu, l'analyse se raffinera sur l'implémentation concrète d'une attaque sur le cryptosystème LWE utilisant le modèle de programmation en nombres entiers analogue à LWE avancé par Shirase [13]. Nous dirigerons naturellement notre recherche vers les ouvertures qui y sont proposées, soit celle de l'évaluation de la complexité du modèle formalisé et de son application au problème Module-LWE, soit le problème le plus utilisé en pratique. Ce travail se terminera ensuite par une réflexion quant à des manières d'optimiser le modèle proposé.

Somme toute, nous avons donc comme objectif de fournir une introduction concise aux cryptosystèmes basés sur les réseaux, ainsi que d'en attaquer un via la programmation en nombres entiers tout en quantifiant ses limites. Ces contributions pourront alors servir de base à de la recherche future, permettant une accessibilité plus universelle à l'attaque de problèmes cryptographiques d'actualité. En effet, le solveur, puisque directement implémenté en Python et exécutable sur une machine personnelle, sera ensuite mis à la disposition de la communauté cryptographique.

Ce rapport débutera par un bref aperçu de la revue de littérature réalisée. Les prérequis tant mathématiques que cryptographiques jugés essentiels à la compréhension adéquate du travail seront définis. Nous proposerons ensuite notre implémentation d'un solveur de problèmes d'optimisation analogues à la modélisation de Shirase. Les résultats de ce solveur à des tests de performance effectués sur des instances de LWE aux paramètres variés seront exposés : cela nous permettra d'offrir une analyse des capacités initiales du modèle. Nous exhiberons ensuite une manière de réduire les problèmes Module-LWE en problèmes Search-LWE, ce qui conclura alors nos réponses aux pistes de recherche avancées par Shirase. En outre, nous partagerons une construction algébrique alternative au problème Search-LWE. Celle-ci nous guidera vers un autre modèle plus puissant, que nous analyserons encore une fois. Nous comparerons ensuite les performances du modèle de Shirase avec celles de notre modèle alternatif.

1 Notions préliminaires

Définition 1. (Réseau euclidien) Soit $\mathbf{b}_1, \dots, \mathbf{b}_n \in \mathbb{R}^m$ un ensemble de vecteurs linéairement indépendants. Le *réseau euclidien* L correspondant à ces vecteurs, que l'on appellera simplement réseau, est l'ensemble des combinaisons linéaires de $\mathbf{b}_1, \dots, \mathbf{b}_n$ à coefficients en $\mathbb{Z}$. On a donc

$$L = \{a_1 \mathbf{b}_1 + a_2 \mathbf{b}_2 + \dots + a_n \mathbf{b}_n : a_1, a_2, \dots, a_n \in \mathbb{Z}\}.$$

On dira que l'ensemble ordonné de vecteurs $B = \{\mathbf{b}_1, \dots, \mathbf{b}_n\}$ est une *base* du réseau L et on écrira alors $L = L(B)$.

Exemple 1. Pour illustrer la structure d'un réseau, considérons un exemple dans $\mathbb{R}^2$. Définissons une base $B = \{\mathbf{b}_1, \mathbf{b}_2\}$ constituée des deux vecteurs linéairement indépendants $\mathbf{b}_1 = (2, 0)^\top$ et $\mathbf{b}_2 = (0, 2)^\top$. Le réseau $L(B)$ engendré par cette base peut alors être visualisé par la grille suivante :

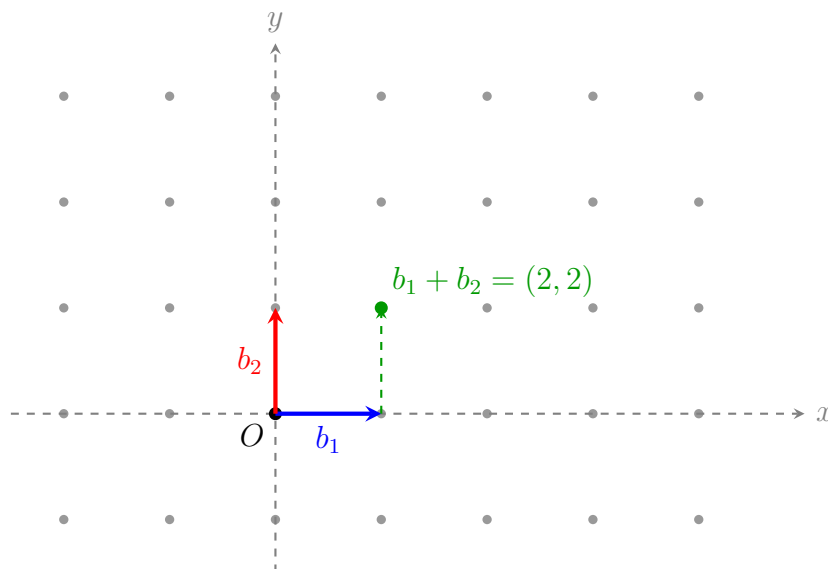


FIGURE 1 – Représentation géométrique du réseau $\mathcal{L}(B)$ engendré par b_1 et b_2 ¹.

1. Notons qu'il s'étendrait à l'infini si des contraintes d'espace ne nous étaient pas imposées.

On peut constater que la base proposée n'est pas la seule base qui génère ce réseau. Par exemple, la base $B' = \{b_3, b_4\}$ constituée des vecteurs $\mathbf{b}_3 = (2, 2)^\top$ et $\mathbf{b}_4 = (2, 4)^\top$ engendre exactement la même grille de points. Avançons que deux bases engendrant un même réseau L sont reliées par une matrice à coefficients entiers que l'on appellera *matrice de passage*, dénotée par U .

Le problème mathématique que nous étudierons dans ce rapport se nomme **LWE**. Définissons-le [11], proposons un exemple [8], et tissons un lien avec les réseaux.

Définition 2. Le problème *Learning With Errors*, que l'on notera $\text{LWE}(m, n, q, B)$, est défini comme suit. Choisissons une matrice $A \in_R \mathbb{Z}_q^{m \times n}$, un vecteur secret $\mathbf{s} \in_R \mathbb{Z}_q^n$ et un vecteur d'erreur $\mathbf{e} \in_R [-B, B]^m$. Posons ensuite $\mathbf{b} = A\mathbf{s} + \mathbf{e} \pmod{q} \in \mathbb{Z}_q^m$. Avec la paire $(A, \mathbf{b})$, on cherche à retrouver $\mathbf{s}$. On qualifiera m comme le nombre «d'échantillons» du problème LWE.

Exemple 2. Soit un problème $\text{LWE}(m, n, q, B)$ avec $m = 5, n = 3, q = 47, B = 2$. On génère donc la matrice A aléatoirement, tandis que $\mathbf{s}$ et $\mathbf{e}$ sont échantillonnés uniformément à partir de $\mathbb{Z}_q^n$ et $[-B, B]^m$ respectivement². Après avoir calculé la valeur de $\mathbf{b}$, on divulgue le couple $(A, \mathbf{b})$ publiquement, ainsi que les valeurs propres au problème que l'on a posées plus tôt. On a donc

$$A = \begin{bmatrix} 27 & 13 & 13 \\ 1 & 46 & 23 \\ 16 & 13 & 30 \\ 18 & 16 & 19 \\ 22 & 0 & 3 \end{bmatrix} \in \mathbb{Z}_{47}^{5 \times 3}, \quad \mathbf{b} = \begin{bmatrix} 30 \\ 28 \\ 25 \\ 34 \\ 32 \end{bmatrix} \in \mathbb{Z}_{47}^5.$$

À travers cette instance précise, le but est donc de trouver le vecteur de solution $\mathbf{s} \in \mathbb{Z}_{47}^3$ et le vecteur de perturbation $\mathbf{e} \in [-2, 2]^5$ qui satisfont $A\mathbf{s} + \mathbf{e} = \mathbf{b} \pmod{47}$. On remarque qu'il y a trois solutions possibles : $\mathbf{s} = (2, 15, 12), \mathbf{e} = (1, 0, 2, 0, -1)$; $\mathbf{s} = (19, 34, 12), \mathbf{e} = (0, 2, 0, 1, 1)$; et $\mathbf{s} = (41, 33, 7), \mathbf{e} = (1, 0, -1, -2, 2)$.

2. Remarquons que cela n'est pas toujours le cas. Nombreuses sont les implémentations de LWE où $\mathbf{s}$ et $\mathbf{e}$ sont échantillonnés selon une distribution précise, telle que la gaussienne discrète ou bien la binomiale centrée.

Définition 3. Soit un problème $\text{LWE}(m, n, q, B)$ et son instance $(A, \mathbf{b})$. Le *réseau* LWE associé à ceux-ci est défini par

$$L_A = \{\mathbf{y} \in \mathbb{Z}^m : \mathbf{y} = A\mathbf{z} \pmod{q} \text{ pour un } \mathbf{z} \in \mathbb{Z}^n\}.$$

On peut voir ce réseau comme l'ensemble de tous les vecteurs $\mathbf{y}$ de dimension m que l'on peut générer depuis la multiplication entre la matrice A et un vecteur secret quelconque $\mathbf{z} \in \mathbb{Z}^n$, le tout étant pris $\pmod{q}$. Le vecteur d'erreur $\mathbf{e}$ n'est donc pas directement considéré.

Des problèmes fondamentaux sont associés aux réseaux euclidiens : présentons les deux plus utiles à notre analyse.

Définition 4. (CVP) Le problème du vecteur le plus proche, abrégé à CVP pour *Closest Vector Problem*, est le problème de recherche du vecteur $\mathbf{v} \in L$ le plus proche d'un autre vecteur $\mathbf{b} \in \mathbb{R}^m$ donné qui n'est pas dans L . Autrement dit, on cherche le vecteur v qui minimise $\|\mathbf{b} - \mathbf{v}\|$.

Définition 5. (BDD_α) Le problème de *décodage à distance bornée*, que l'on note BDD_α , est défini tel que l'on a $L \subseteq \mathbb{R}^m$ un réseau et $\mathbf{b} \in \mathbb{R}^m$ un vecteur avec la garantie qu'il existe un unique vecteur $\mathbf{v} \in L$ à distance α de $\mathbf{b}$. On cherche à trouver ce $\mathbf{v}$.

Remarquons que ces deux problèmes sont extrêmement similaires : BDD spécifie simplement le rayon de recherche du vecteur $\mathbf{v}$ autour du vecteur $\mathbf{b}$ en offrant une garantie d'unicité. CVP est considéré comme un problème très général et fondamental au niveau des réseaux, tandis que BDD agit un peu comme l'une de ses variantes.

Concluons cette section avec une toute petite introduction à l'optimisation.

Définition 6. (Problème PQNE) Le *problème quadratique de programmation en nombres entiers*, pour lequel nous utiliserons l'acronyme PQNE, est défini comme le suivant :

$$\max\{x^T Q x + c x : A x \leq b, x \in \mathbb{Z}_+^n\}.$$

Dans celui-ci, la fonction à maximiser contient des termes quadratiques, sous les contraintes $Ax \leq b$, pour laquelle il faut s'assurer que les variables x soient entières et positives. Naturellement, nous proposons du même coup l'acronyme PNE pour *programmation en nombres entiers*.

Lorsque nous évaluerons la performance de différents modèles d'optimisation, nous les quantifierons parfois selon le nombre de noeuds qu'ils auront explorés. Ces noeuds découlent directement de l'algorithme *Branch-and-Bound*, soit l'un des algorithmes principaux utilisés en PNE [2].

2 État de l'art

Nombreuses sont les techniques de casse ayant été proposées pour le cryptosystème LWE. Le challenge *Learning With Errors* de l'Université de technologie de Darmstadt [16] propose des instances variées du problème : les cryptologues universitaires soumettent alors leurs solutions à des instances de taille grandissante, tout en présentant du même coup l'algorithme qu'ils ont utilisé afin d'arriver à ce succès.

Il est alors intéressant de remarquer que la trentaine de solutions exposées dans le «temple de la renommée» du problème utilisent toutes à un certain moment un algorithme de réduction de réseau quelconque. Celles-ci découlent toutes de l'algorithme LLL, pour Lenstra, Lenstra et Lovász [4]. Proposons son fonctionnement en grandes lignes.

Définition 7. Le processus de Gram-Schmidt prend une base $\mathcal{B} = \{\mathbf{v}_1, \dots, \mathbf{v}_n\}$ en entrée et retourne une base $\mathcal{B}^* = \{\mathbf{v}_1^*, \dots, \mathbf{v}_n^*\}$ parfaitement orthogonale. L'espace continu engendré par $\mathcal{B}$ et par $\mathcal{B}^*$ est le même par définition du procédé, cependant, les réseaux discrets correspondant à ceux-ci ne sont pas les mêmes, i.e. $L(\mathcal{B}) \neq L(\mathcal{B}^*)$. $\mathcal{B}^*$ est donc un référent idéal sur lequel sera basé *LLL*

Définition 8. Soit $\mathcal{B}$ une base quelconque d'un réseau L et soit $\mathcal{B}^*$ la base de Gram-Schmidt qui y est associée. Alors $\mathcal{B}$ est dite *LLL-réduite* si et seulement si elle satisfait les deux conditions suivantes :

1. **Condition de taille**

$$|\mu_{i,j}| = \frac{|\mathbf{v}_i \cdot \mathbf{v}_j^*|}{\|\mathbf{v}_j^*\|^2} \leq \frac{1}{2}$$

pour tout $1 \leq j < i \leq n$.

2. **Condition de Lovász**

$$\|\mathbf{v}_i^*\|^2 \geq \left(\frac{3}{4} - \mu_{i,i-1}^2\right) \|\mathbf{v}_{i-1}^*\|^2$$

pour tout $1 < i \leq n$.

De ces conditions découlent alors le pseudocode de l'algorithme :

Algorithme 1 : Algorithme LLL

Entrée : Une base $\mathcal{B} = \{\mathbf{v}_1, \dots, \mathbf{v}_n\}$ d'un réseau L

Sortie : Une base LLL-réduite $\mathcal{B} = \{\mathbf{v}_1, \dots, \mathbf{v}_n\}$

$\mathbf{v}_1^* \leftarrow \mathbf{v}_1$;

$k \leftarrow 2$;

tant que $k \leq n$ **faire**

pour $j = k - 1$ **à** 1 **faire**

$\mathbf{v}_k \leftarrow \mathbf{v}_k - \lfloor \mu_{k,j} \rfloor \mathbf{v}_j$;

si *Condition de Lovász validée* **alors**

$k \leftarrow k + 1$

sinon

Échanger $\mathbf{v}_{k-1}$ et $\mathbf{v}_k$;

$k \leftarrow \max(k - 1, 2)$

Retourner $\{\mathbf{v}_1, \dots, \mathbf{v}_n\}$;

Nous avons montré plus tôt qu'il était possible que deux bases génèrent le même espace. Justement, l'algorithme LLL prend une base que l'on peut qualifier de «mauvaise», soit une base où les vecteurs sont presque linéairement dépendants et sont de très grande taille, et retourne une «bonne» base, soit une base où les vecteurs sont plus courts et où les vecteurs sont plus orthogonaux. Cette transformation vient toujours faciliter les méthodes de résolution théorisées : c'est avec de petits vecteurs qu'elles deviennent plus optimales. En pratique, c'est plutôt l'algorithme BKZ qui est

implémenté. Il est possible de considérer BKZ comme un LLL travaillant par blocs : leur fonctionnement est quasi identique. En changeant la taille des blocs sur lesquels travaille BKZ, nous obtenons généralement une meilleure probabilité de casse de grandes instances de LWE. Cependant, cela vient avec un coût énorme en ressources computationnelles.

Il est donc rare de qualifier directement la vitesse d'un solveur LWE, car celle-ci dépend beaucoup plus de la force de l'algorithme de réduction de réseau utilisé que de la couche finale de résolution elle-même. Par exemple, pour un $n = 55$, un $q = 3037$ et un $B = 75$, cela pris 83 jours RTX 4090 : cela est équivalent à une carte graphique roulant pendant 83 jours, ou 83 cartes graphiques sur un seul jour. La cassure de grandes instances de LWE suit toujours cette logique : nous obtenons alors une variation de la vitesse de casse selon la quantité de matériel informatique disponible, car l'algorithme BKZ, que l'on peut considérer comme l'état de l'art, est très demandant en ressources.

Note. Tel que mentionné dans l'introduction, les sections 1 et 2 ne couvrent que les prérequis nécessaires à la compréhension de ce rapport. La revue de littérature plus détaillée est disponible en [5].

3 Méthodologie

Notons pour commencer que les prérequis mathématiques à ce projet étaient assez lourds. La première partie du projet fut alors principalement consacrée à de la lecture : les concepts abordés dépassaient ce qui est enseigné dans les cours du baccalauréat. Bien que cela ne soit pas de la recherche nouvelle, rappelons qu’une revue de littérature [5] fut donc rédigée. Par souci de propreté, le système de composition de document LaTeX fut utilisé afin de mener à bien cette rédaction. Le travail présenté dans ce rapport ne traite pas explicitement de la revue de littérature, mais notons que sa qualité pédagogique sera aussi évaluée en la proposant à d’autres étudiants.

Le coeur de la recherche vint donc de la liaison audacieuse entre l’optimisation et la cryptographie. En effet, la programmation en nombres entiers n’a jamais été directement utilisée pour résoudre des instances du problème LWE . Cependant, en 2023, Shirase [13] avance pour la première fois une réduction depuis LWE vers un problème de PQNE . Cette dernière n’a jamais été implémentée : nous cherchons donc à la tester empiriquement. Notre recherche aborde donc initialement le problème d’un angle plus technique et informatique, les modélisations mathématiques ayant déjà été proposées.

À cette fin, nous proposons la conception d’un solveur codé via Gurobi, un solveur d’optimisation directement utilisable depuis une bibliothèque Python. Gurobi est souvent considéré comme étant le solveur à la fine pointe de la technologie. Heureusement, il nous est possible d’obtenir une licence académique associée à l’Université de Montréal.

La modélisation n’étant pas la nôtre, nous évaluerons la qualité de ce solveur initial selon sa justesse d’implémentation et sa capacité à retourner de bons résultats. Tout de même, des tests de performances seront proposés. Cela nous permettra de repérer les lacunes de ce dernier, afin de laisser le projet prendre une tournure plus mathématique : notre recherche s’axera ensuite sur l’amélioration du modèle proposé par Shirase. Nous nous fierons à de la théorie d’optimisation et à d’autres techniques de résolution de LWE afin d’adapter le modèle initial. Nous évaluerons ensuite em-

piriquement l'impact qu'auront les changements proposés : toute amélioration sera alors la bienvenue, compte tenu qu'aucune recherche n'a été directement effectuée sur ce modèle.

Pour terminer, nous analyserons la performance des multiples solveurs évalués face aux standards cryptographiques actuels. Il est difficile de s'attendre à un succès³ : il sera donc pertinent de comparer leur capacité face aux algorithmes de l'état de l'art. Notons que les bibliothèques Python FPylll (pour l'implémentation de l'algorithme LLL), GurobiPY (pour Gurobi), numpy et sympy (pour les calculs numériques) et pandas (pour l'analyse et la structuration de données) seront utilisées.

3. Si nous brisons Kyber, une implémentation de LWE, avec un ordinateur portable, cela serait fort probablement la plus grande contribution cryptographique de la décennie.

4 Modélisation initiale

4.1 Cadre mathématique et implémentation

Dans un article de 2023, Shirase [13] réduit le problème Search-LWE au problème de PQNE. Le problème Search-LWE est exactement le problème que nous avons posé en définition 2 : c'est la version de base de LWE. Rappelons qu'il pose la relation

$$As + \mathbf{e} \equiv \mathbf{b} \pmod{q}$$

depuis les valeurs $A \in \mathbb{Z}_q^{m \times n}$, $\mathbf{s} \in \mathbb{Z}_q^n$, $\mathbf{e} \in \mathbb{Z}^m$ et $\mathbf{b} \in \mathbb{Z}_q^m$. Dans le cas précis de la réduction, Shirase travaille avec un vecteur $\mathbf{e}$ tel que ses composantes sont choisies selon une loi $N(0, \sigma^2)$, soit une normale centrée sur 0 avec écart-type σ . On rappelle que le but de cette construction, et donc le but de LWE, est de déterminer $\mathbf{s}$ depuis le couple $(A, \mathbf{b})$.

Afin de s'aligner avec le travail de Shirase, redéfinissons $A, \mathbf{b}, \mathbf{e}$ en introduisant de nouvelles variables. Nous posons :

$$A = \begin{pmatrix} A_0 \\ A_1 \end{pmatrix}, \quad \text{où} \quad \begin{cases} A_0 \text{ est la matrice } n \times n \text{ supérieure de } A, \\ A_1 \text{ est la matrice } (m - n) \times n \text{ inférieure de } A. \end{cases}$$

$$\mathbf{b} = \begin{pmatrix} \mathbf{b}_0 \\ \mathbf{b}_1 \end{pmatrix}, \quad \text{où} \quad \begin{cases} \mathbf{b}_0 \text{ est le vecteur colonne de dimension } n \text{ supérieur de } \mathbf{b}, \\ \mathbf{b}_1 \text{ est le vecteur colonne de dimension } m - n \text{ inférieur de } \mathbf{b}. \end{cases}$$

$$\mathbf{e} = \begin{pmatrix} \mathbf{e}_0 \\ \mathbf{e}_1 \end{pmatrix}, \quad \text{où} \quad \begin{cases} \mathbf{e}_0 \text{ est le vecteur d'erreur de dimension } n \text{ supérieur de } \mathbf{e}, \\ \mathbf{e}_1 \text{ est le vecteur d'erreur de dimension } m - n \text{ inférieur de } \mathbf{e}. \end{cases}$$

Remarque 1. Pour le reste de notre étude, nous simplifierons la notation utilisée. Lorsque nous travaillons en $(\text{mod } q)$, on abrégera $a \equiv b \pmod{q}$ à $a \equiv b$. Pour deux vecteurs $\mathbf{v} = (v_1, v_2, \dots, v_n)$ et $\mathbf{w} = (w_1, w_2, \dots, w_n)$, on notera $\mathbf{v} \equiv \mathbf{w}$ si $v_i \equiv w_i \forall i$. Pour deux matrices $A = (a_{i,j})$ et $B = (b_{i,j})$, on notera $A \equiv B$ si $a_{i,j} \equiv b_{i,j} \forall i, j$.

Nous ne présenterons pas la méthode entière de Shirase. Notons par contre qu'il pose les contraintes au modèle de PQNE sous les i équations suivantes :

$$w_{i,1}e_1 + w_{i,2}e_2 + \dots + w_{i,n}e_n - e_{n+i} + qf_i = -u_i \quad (1)$$

et qu'il est possible de fixer un $t \in \mathbb{N}$ tel que

$$-t \leq e_i \leq t$$

selon la distribution utilisée pour l'erreur. Shirase pose ensuite des bornes sur les f_i via l'équation 1, et introduit pour terminer le lemme suivant proposant de nouvelles quantités.

Lemme 1. Soit $(A, \mathbf{b})$ une instance LWE, où $A \in \mathbb{Z}_q^{m \times n}$ et $\mathbf{b} \in \mathbb{Z}_q^m$. On pose A_0 et $\mathbf{b}_0$ tel que nous l'avons fait plus tôt. Alors :

1. Il est possible de poser une matrice W de taille $(m - n) \times n$ où $W = (w_{i,j})$ en utilisant les $w_{i,j}$ nos contraintes. Pour la calculer, on a $W \equiv A_1 A_0^{-1}$. Il s'agit d'un calcul rapide.
2. Il est possible de calculer le vecteur entier de u_i tel que posé par nos contraintes via $(u_1, u_2, \dots, u_{m-n})^T \equiv \mathbf{b}_1 - A_1 A_0^{-1} \mathbf{b}_0$.

Cela nous permet alors de proposer notre réduction complète.

Théorème 1. (Réduction de LWE à PNE)

Soit $(A, \mathbf{b})$ une instance LWE, où $A \in \mathbb{Z}_q^{m \times n}$ avec $n < m$, et $\mathbf{b} \in \mathbb{Z}_q^m$. On pose A_0 et $\mathbf{b}_0$ tel que nous l'avons fait plus tôt. Les variables $w_{i,j}$ et u_i avec $i = 1, \dots, m - n$; $j = 1, \dots, n$ sont calculées telles que dans le lemme précédent, et une valeur pour le paramètre $t \in \mathbb{N}$ est choisie. Il est alors possible de construire le problème de programmation en nombres entiers présenté en début de page suivante.

La théorie finale étant posée, donnons un aperçu de l'architecture de notre solution permettant l'implémentation et le test de cette modélisation⁴. Le fichier le plus

4. Le code référencé est accessible en [6]

$$\begin{aligned}
\text{minimiser : } & \sum_{k=1}^m x_k^2 \\
\text{sujet à : } & \sum_{j=1}^n w_{i,j} x_j - x_{n+i} + q f_i = -u_i, & \forall i \in \{1, 2, \dots, m-n\} \\
& -t \leq x_k \leq t, & \forall k \in \{1, 2, \dots, m\} \\
& -\left\lfloor \frac{t(nq - n + 1) + q - 1}{q} \right\rfloor \leq f_i \leq \left\lfloor \frac{t(nq - n + 1)}{q} \right\rfloor, & \forall i \in \{1, 2, \dots, m-n\} \\
& x_k, f_i \in \mathbb{Z}
\end{aligned}$$

FIGURE 2 – Modèle de PQNE pour la résolution de Search-LWE.

important, soit le fichier d'implémentation *solve_lwe*, est notre fichier de résolution de base basé sur l'article de Shirase. Nous l'avons donc glissé en annexe 6.3. Il prend en entrée une instance $(A, \mathbf{b})$ de LWE, ainsi que son paramètre q , régissant l'espace $\mathbb{Z}_q$ sur lequel on travaille. Les paramètres t et *time_limit* sont des paramètres plus «ajustables» : t régit la borne sur notre erreur, tandis que *time_limit* met une limite de temps au modèle pour qu'il ne s'éternise pas sur une instance excessivement longue à résoudre. S'il arrive à trouver la solution à l'instance correspondant aux paramètres d'entrées, il retournera le vecteur de solution $\hat{\mathbf{s}}$, le vecteur d'erreur $\mathbf{e}$, ainsi que le nombre de «nodes» de Branch-and-Bound explorées par Gurobi et le temps de résolution du problème. Autrement, le solveur retourne des valeurs vides, ainsi que le nombre de nodes et le temps de résolution du problème.

Il était alors impératif de vérifier la validité du solveur avant de faire des tests plus poussés sur ce dernier : cela nous confirmerait de la justesse du modèle de Shirase. Un tout petit exemple numérique, tiré de la littérature [12], a donc été employé. Celui-ci peut être aperçu dans le fichier *test_manuel*, et il a été passé avec brio.

Nous avons ensuite créé un fichier de génération : ce dernier contient toutes les fonctions que l'on désire appeler pour à la fois générer des paramètres et des instances. Forcément, les générateurs d'instances utilisent les générateurs de paramètres pour

créer un problème LWE valide. C’est un fichier appelé *benchmark* qui consolide ces générateurs et qui envoie ensuite les instances créées au solveur. *Benchmark* compare ensuite la solution théorique à la solution trouvée par le solveur via une fonction *test_auto*, et retourne ensuite les informations pertinentes aux tests effectués dans un fichier CSV.

4.2 Difficultés rencontrées

Plusieurs petites embûches se mirent dans notre chemin lorsque fut le temps d’exécuter le code pour la première fois. Pour commencer, remarquons que le théorème 1 avance l’utilisation de l’inverse matriciel de A_0 . Cependant, cet inverse est calculé mod q , et certaines matrices n’en admettent simplement pas. Il fut donc nécessaire de préévaluer les matrices proposées par nos générateurs d’instances, autrement le solveur nous retournait des erreurs de calcul mathématique.

En second lieu, il fut difficile de déterminer de bonnes bornes t à poser sur les composantes de $\mathbf{e}$. Il nous fallait absolument permettre à notre solveur de déterminer toutes les valeurs de l’erreur possible, autrement nous manquerions des solutions car notre espace de recherche serait trop petit. Cependant, pour une normale avec un écart type de 1, il y a environ une chance sur 150 qu’une composante d’erreur soit de 3 ou plus, mais passer d’un $t = 2$ à un $t = 3$ vient augmenter exponentiellement la difficulté de résolution pour notre solveur. Comme nous exécutons une quantité impressionnante de tests, il est certain qu’une composante d’erreur ne soit pas trouvable un jour, ce qui est un aléa inadmissible à l’évaluation de tests fiables. Comme nous désirions rester fidèle à la théorie de Shirase, nous avons alors implémenté une distribution normale tronquée, nous permettant de générer les valeurs selon une gaussienne sans détruire la performance de notre solveur : ce dernier n’avait alors pas besoin d’aller chercher des valeurs à probabilité très faible d’exister⁵.

Le conditionnement des problèmes LWE fut aussi un enjeu mathématique que nous dûmes prendre en compte. Le modèle de Shirase argumente via les propriétés

5. Notons que les implémentations pratiques de LWE n’utilisent que très rarement une gaussienne pour cette raison. Nous présenterons une distribution alternative plus tard.

de la distribution gaussienne que la solution à un problème LWE sera toujours celle où la norme du vecteur d'erreur $\mathbf{e}$ est minimale. Indirectement, il modélise alors une attaque au problème du CVP. Cela entend que même en ne bornant pas $\mathbf{e}$, nous retrouverions quand même une solution optimale dans un monde où il existe plusieurs solutions. Empiriquement, cela serait excessivement long pour Gurobi : il comparerait chaque solution trouvée avec les centaines voire milliers de solutions possibles. Justement, pour pallier ce problème, Shirase ajoute une borne t sur les composantes de l'erreur, sans vraiment la justifier. Celle-ci vient alors transformer le problème de CVP en un problème de BDD_α où $\alpha = t\sqrt{m}$, soit la norme euclidienne maximale possible pour $\mathbf{e}$. Selon Menezes [8], la probabilité qu'un système LWE admette plus d'une solution dans un espace borné par t est calculable via

$$(q^n - 1) \left(\frac{4t + 1}{q} \right)^m. \quad (2)$$

Nous nous sommes donc assurés que cette probabilité reste minime dans notre génération d'instances, ce qui nous permettait de terminer par défaut l'exécution de Gurobi dès qu'il trouve une solution. Les cryptosystèmes basés sur LWE conditionnent toujours leurs systèmes de cette manière, ce qui justifie ici notre démarche.

La dernière difficulté que nous avons rencontré fut celle de la gestion de l'aléa dans nos tests de performance. Nous générions nos instances aléatoirement, ce qui fit que pour de mêmes quadruplets de paramètres, nous obtenions des instances aux temps de résolution complètement différents. L'ajustement fut assez rapide : nous avons alors commencé à fixer des «seeds» à nos tests, ce qui nous permit de réutiliser de manière plus adéquate les instances que nous générions.

4.3 Résultats préliminaires

Pour cette section de tests, nous travaillons sur une machine Windows utilisant Gurobi 13.0.2. Nous utilisons les 12 threads d'un Ryzen 5 8645HS d'AMD avec 16 Go de RAM. Les résultats sont extirpés depuis le fichier CSV produit par notre fichier benchmark, et sont ensuite transformés en graphiques via matplotlib.

En premier lieu, une série de tests bien conditionnés ont été effectuées. Dans ces tests, le paramètre n était fixé à 10, le paramètre q à 101, tandis que le paramètre m était graduellement incrémenté. Pour ce qui est de t , il respectait l'équation 2 bornée par une probabilité de 2^{-12} , et donc était aussi incrémenté au fur et à mesure qu'augmentait m . On peut donc le voir comme un test dynamique : deux paramètres variaient simultanément. Nous pouvons introduire cette variation sur les graphiques ci-dessous ⁶ :

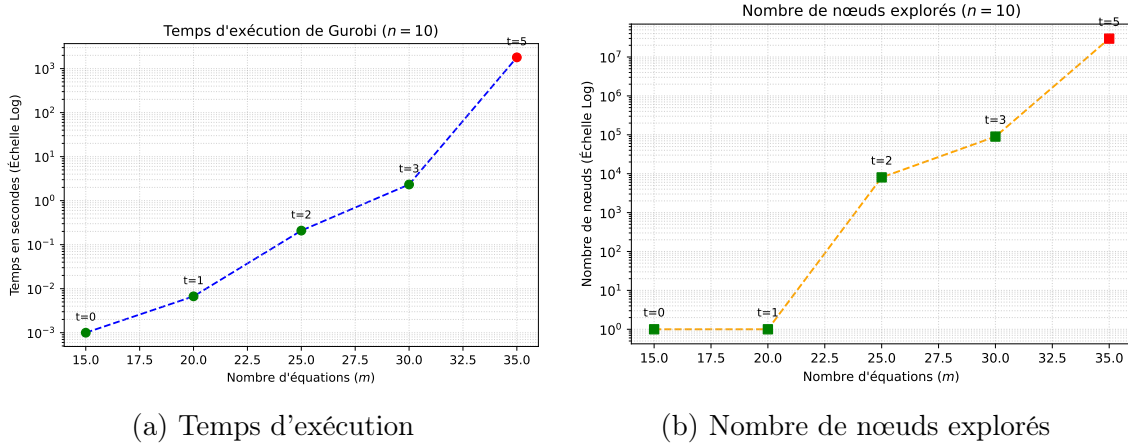


FIGURE 3 – Analyse de la performance de Gurobi pour m, t croissant et $n = 10$.

Directement, il nous est possible de confirmer que la complexité de résolution de notre modèle LWE est exponentielle selon les paramètres t et m . En produisant des graphiques à échelle logarithmique, nous créons des relations avec un semblant de linéarité. Par défaut, nous avons limité le temps d'exécution de Gurobi à 30 minutes, soit 1800 secondes : nous pouvons apercevoir que le couple $m = 35, t = 5$ n'a pas terminé son exécution dans les temps impartis. Somme toute, ces graphiques nous permettent de comprendre que notre modèle a pour l'instant ses limites : il ne peut résoudre de grandes instances du problème LWE.

Naturellement, il suivait de créer ensuite un test où l'on ferait varier uniquement

6. À titre de convention, nos cercles sont verts si Gurobi a trouvé la bonne solution, tandis qu'ils sont rouges s'il n'a pas trouvé la solution, donc si la solution qu'il a trouvée est mauvaise ou qu'il n'en a pas trouvée une.

le paramètre n . Rappelons que c'est n qui encode la taille du secret dans les instances LWE : l'analyse de performance d'une attaque est souvent évaluée selon celui-ci. Nous avons donc fixé $m = 20, q = 149$ et $t = 2$.

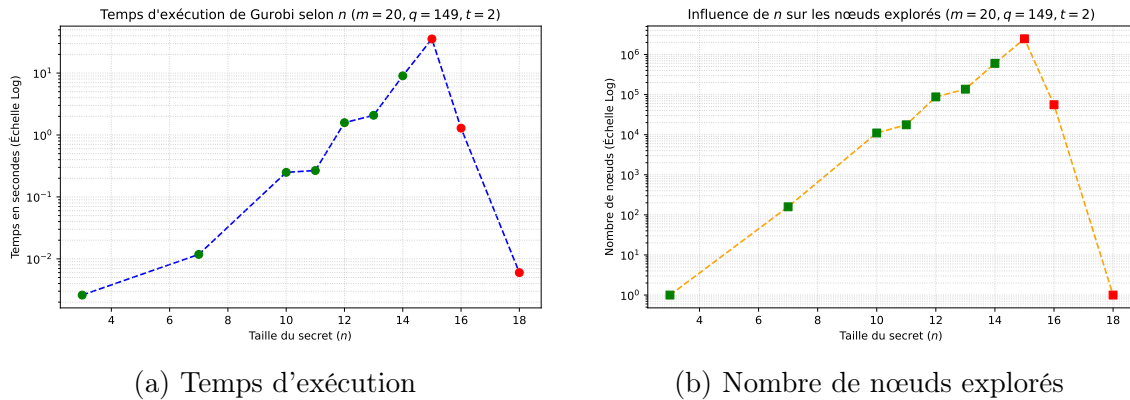


FIGURE 4 – Influence du paramètre n sur la performance pour $m = 20$ et $t = 2$.

Remarquons que nos résultats sont encore une fois fidèles à la théorie : le temps de calcul augmente exponentiellement lorsque la taille du secret n augmente linéairement : c'est de là que provient la difficulté du problème LWE. Cependant, remarquons aussi quelque chose d'autre d'intéressant : lorsque n arrive dans la zone critique près de la valeur de m , Gurobi ne retourne pas la bonne solution : en effet, nous arrivons alors dans des systèmes mal conditionnés à haute probabilité selon l'équation 2. Remarquons que le calcul de cette mauvaise solution est de plus en plus rapide : en effet, plus n se rapproche de m , plus le nombre de mauvaises solutions est grand, et donc plus il est rapide de tomber sur une solution quelconque. En $n = 18$, on trouve donc cette mauvaise solution beaucoup plus efficacement qu'en $n = 15$.

En continuant nos tests sur des paramètres particuliers, il était nécessaire d'analyser l'impact de la valeur de t sur le solveur : le graphique ci-dessous présente les résultats retournés. Nous travaillons ici avec un $q = 101$ et un $m = 35$. Remarquons pour commencer que l'augmentation de la borne d'erreur t semble poser plus de problèmes à Gurobi que celle du paramètre n : pour $t = 3, 4$, le solveur tombait constamment en arrêt forcé après 20 minutes pour un système de plus de 11

équations. Cela est fidèle à la théorie : l'augmentation de ce paramètre vient faire grandir exponentiellement l'espace de recherche du problème. Remarquons aussi que le graphique proposé explicite bien les difficultés que comporte l'analyse de performance d'attaque de cryptosystèmes modernes. Sans équipement spécialisé, le «mur» matériel est vite atteint. L'ajustement tout léger d'un seul paramètre peut vite faire passer un problème d'instantanément résoluble à résoluble en plus d'une heure. Il devient alors difficile de simuler plusieurs cours d'essais.

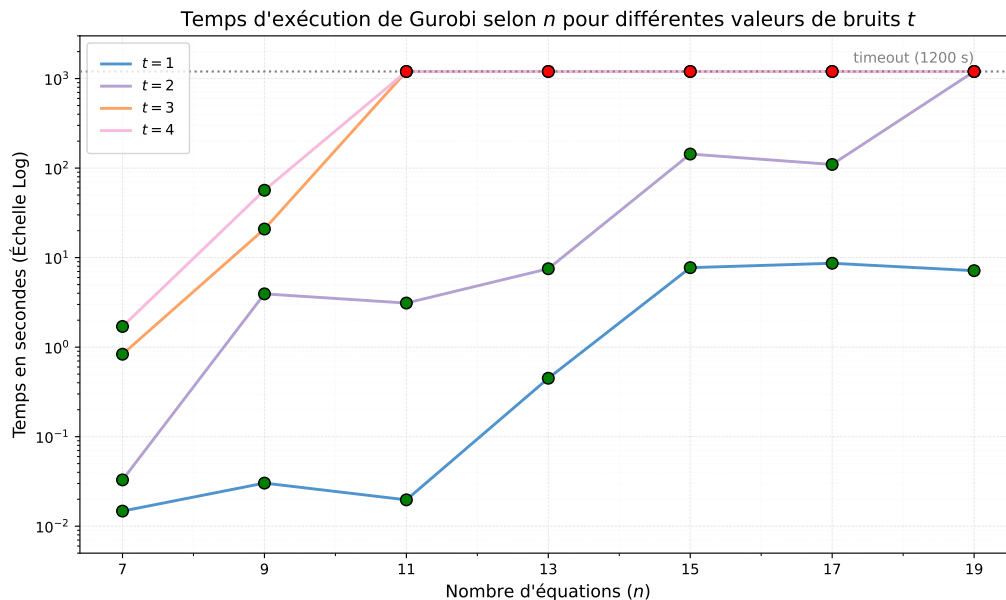


FIGURE 5 – Évolution du temps de calcul en fonction de m , selon un t variable.

En guise d'observation générale, avançons que notre modèle initial est de complexité exponentielle selon les paramètres n et t , et que $m \approx 35$, $n \approx 20$, $q \geq 101$ et un $t \geq 2$ constituent ses limites. Nous utiliserons ces valeurs de référence lorsque nous tenterons d'évaluer l'impact des améliorations que nous apporterons à ce dernier.

5 Cryptosystèmes actuels et réduction $\text{MLWE} \leq \text{LWE}$

Justifions la nécessité d'améliorer notre modèle initial en comparant les dimensions des instances de chiffrement utilisées par les implémentations concrètes de LWE. Le cryptosystème Kyber est considéré comme le prochain standard permettant l'échange de clés, remplaçant RSA [9]. C'est donc celui-ci que nous analyserons. Remarquons que Kyber n'utilise pas la version classique de *learning with errors*, mais plutôt le problème Module-LWE. La réduction de Shirase ne s'appliquant qu'au problème de base, il n'est pas possible d'établir une relation directe entre Kyber et notre modèle de PQNE. Cependant, il est possible de les rapprocher : commençons donc par présenter Module-LWE.

Définition 9. (Module-LWE) Le problème *Module-LWE*, que l'on abrège à MLWE et qui prend en entrée le quintuplet (k, l, n, q, B) , se pose comme suit. Soit $A \in_R R_q^{k \times l}$, $\mathbf{s} \in_R R_q^l$ et $\mathbf{e} \in_R S_B^k$, avec $k > l$, $B \ll q/2$ et $R_q = \mathbb{Z}_q[x]/\langle x^n + 1 \rangle$. En ne connaissant que A et $\mathbf{b} = A\mathbf{s} + \mathbf{e} \in R_q^k$, le but du problème est de trouver $\mathbf{s}$.

Pour ne pas débiter de discussion abstraite sur la théorie des anneaux, contentons-nous de dire que $R_q = \mathbb{Z}_q[x]/\langle x^n + 1 \rangle$ est l'ensemble de tous les polynômes de degré inférieur à n dont les coefficients sont dans $\mathbb{Z}_q$. De plus, voyons simplement S_B comme l'ensemble des polynômes dont les coefficients sont bornés par B et dont le degré est aussi strictement plus petit que n .

Depuis cette définition, remarquons que la matrice A dans MLWE est une matrice constituée de polynômes. Heureusement, il est possible de la transformer en matrice composée uniquement d'entiers compris dans $\mathbb{Z}_q$ pour qu'elle devienne interprétable par Gurobi [8]. Nous présentons cette réduction naïve mais adéquate en section 6.4 de Lessnick [5]. Bien que celle-ci ne soit pas très compliquée, sa preuve est visuellement encombrante due à de grandes représentations matricielles : nous ne voulons pas surcharger ce rapport.

En posant k, l, n, q, B comme les paramètres du problème MLWE de base et en posant sous $\hat{m}, \hat{n}, \hat{q}$ et $\hat{B}$ les paramètres du système LWE classique associé à ce

problème MLWE, nous pouvons affirmer via cette réduction que $\hat{m} = k \cdot n$, que $\hat{n} = l \cdot n$, que $\hat{q} = q$ et que $\hat{B} = B$.

Le cryptosystème Kyber est flexible : il admet trois niveaux de sécurité via ses trois variations, soit ML-KEM-512, ML-KEM-768 et ML-KEM-1024. Dans le tableau ci-dessous, nous présentons les paramètres utilisés par ces dernières.

Algorithme	Niveau NIST	n	q	k	B
ML-KEM-512	1	256	3329	2	3
ML-KEM-768	3	256	3329	3	2
ML-KEM-1024	5	256	3329	4	2

TABLEAU 1 – Paramètres de ML-KEM selon son niveau de sécurité.

Une fois transformées en LWE classique, nous obtenons les paramètres suivants :

Algorithme	$\hat{m} = k \cdot n$	$\hat{n} = l \cdot n$	$\hat{q} = q$	$\hat{B} = B$
ML-KEM-512	512	512	3329	3
ML-KEM-768	768	768	3329	2
ML-KEM-1024	1024	1024	3329	2

TABLEAU 2 – Système LWE classique équivalent aux instances de ML-KEM.

Si l'on voulait simuler une attaque sur une instance de Kyber, il faudrait alors tenter de briser une instance de LWE classique avec des paramètres m et n étant de l'ordre de plus de 500. Pour l'instant, notre solveur semble prendre énormément de temps lorsque les valeurs de m et de n dépassent la trentaine. Un tel écart entre notre taille d'instance limite et entre celles de Kyber semble indiquer que des améliorations pourraient être possibles : les paramètres de Kyber ne seraient pas si élevés si des techniques de déchiffrement alternatives n'étaient pas plus puissantes que la nôtre.

6 Améliorations

Nous utilisons encore une fois Gurobi 13.0.2, sous les 6 des 12 threads d'un Ryzen 5 8645HS d'AMD⁷. Cependant, pour cette section-ci, nous travaillons sur un système WSL, soit un système Linux exécuté directement à l'intérieur de Windows. Ce dernier est initialisé pour utiliser toute la capacité possible de notre processeur, mais seulement 12 des 16 Go de RAM disponibles. La configuration système est donc légèrement réduite par rapport à ce qui a été effectué lors des tests initiaux.

Dans cette section, nous présentons 3 modélisations différant légèrement du modèle initial. Notons que le nombre premier q utilisé sera fixé à 3329, s'alignant ainsi avec Kyber. De plus, nous changeons la distribution gaussienne tronquée que nous utilisons pour l'erreur pour une distribution binomiale centrée bornée (DBC) au paramètre de dispersion n . Il est possible de la définir comme suit [1] :

Échantillonner $(a_1, \dots, a_n, b_1, \dots, b_n) \leftarrow \{0, 1\}^{2n}$

et retourner $\sum_{i=1}^n (a_i - b_i)$.

Cela constitue une manière alternative de contenir les coefficients du vecteur d'erreur $\mathbf{e}$ entre $[-n, n]$, et est l'implémentation pratique en industrie.

6.1 Modélisation CMOD

Une légère mais non-négligeable optimisation qui sera appliquée sur notre modèle de base est celle de l'implémentation du modulo symétrique [8]. Les articles purement théoriques n'en font jamais allusion, mais les systèmes pratiques implémentent toujours leur modulo de cette manière - Kyber n'en fait pas exception. Nous définissons cet opérateur comme suit :

Définition 10. Soit $q \geq 3$ un entier impair, et soit $r \in \mathbb{Z}_q$. On définit le *modulo*

7. Ces derniers sont manuellement limités sous Gurobi : cela semblait offrir plus de performance.

symétrique de r par :

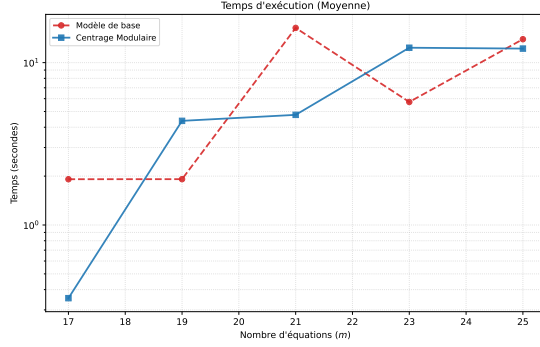
$$r \bmod q = \begin{cases} r & \text{si } r \leq (q-1)/2 \\ r - q & \text{si } r > (q-1)/2 \end{cases}$$

Remarquons que $-(q-1)/2 \leq r \bmod q \leq (q-1)/2$. Par exemple, si $q = 13$ et $r \in \mathbb{Z}_{13}$, alors nous avons $-6 \leq r \bmod 13 \leq 6$. Avec un mod classique, nous aurions $0 \leq r \bmod q \leq 12$. Le nombre d'entiers entre ces bornes reste forcément le même peu importe l'opérateur modulo choisi, mais les valeurs sont beaucoup moins élevées avec mods qu'avec un mod classique.

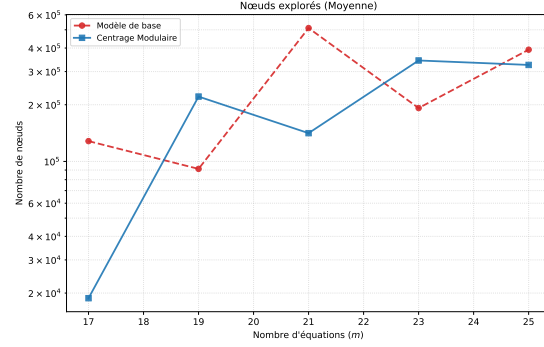
Cette façon de concevoir les modulus est beaucoup plus en lien avec les distributions d'erreurs que nous avons présentées depuis le début : elles sont toutes centrées autour de 0. Par exemple, garder le modulo classique avec une binômiale centrée engendrerait des erreurs conceptuelles. Un coefficient secret quelconque pourrait être très loin de 0, soit à 3328, mais en subissant une erreur de +1, il se retrouverait directement sur 0. Avec un modulo centré, nous passerions de 1664 à -1664 : la représentation de la distance d'un point à l'origine est donc beaucoup plus fidèle à la réalité en modulo centré. Malgré tout, le modèle de Shirase initial à modulo classique ne causait aucun problème d'exactitude. Nous supposons alors que ce changement était seulement d'ordre conceptuel.

À titre de comparaison et de validation, nous avons alors implémenté ce modèle sous Gurobi. La valeur de n a été fixée à 11 et 5 valeurs de m ont été utilisées. Pour chacune d'elles, 30 tests, correspondant aux seeds 1 à 30, nous ont permis d'obtenir un échantillon significatif.

Le graphique que nous présentons ci-dessous présente la performance du solveur *solve_lwe* classique versus celle du solveur *solve_lwe_cmod* implémentant le centrage modulaire. Il semble difficile de tirer une conclusion lorsque nous regardons uniquement les statistiques sur les tests calculés avec la moyenne. Il suffit d'y avoir un test qui prenne énormément de temps pour faire exploser la valeur de cette der-



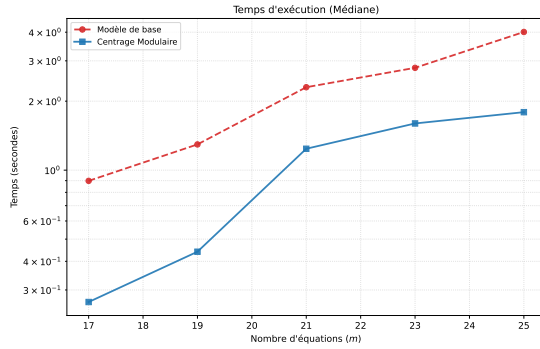
(a) Temps d'exécution



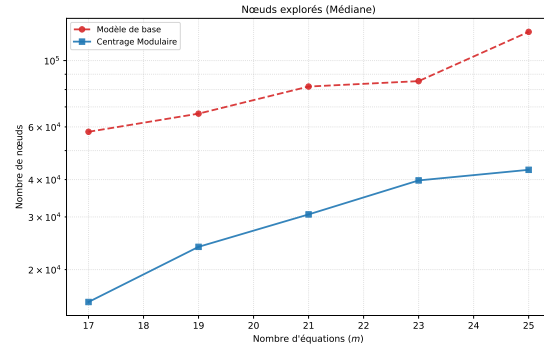
(b) Nombre de nœuds explorés

FIGURE 6 – Comparaison (en moyenne) du modèle de base avec le modèle CMOD.

nière, tant pour les noeuds que pour le temps. Nous avons donc décidé d'extraire aussi les résultats médians.



(a) Temps d'exécution



(b) Nombre de nœuds explorés

FIGURE 7 – Comparaison (médiane) du modèle de base avec le modèle CMOD.

Nous observons alors une amélioration significative suite à l'implémentation du centrage modulaire pour notre solveur : les instances résolues avec le solveur *solve_lwe_cmod* ont leur solution découverte deux à trois fois plus rapidement. Nous supposons que cela découle des variables de compensation f_i . Rappelons que les contraintes

principales du modèle de Shirase prennent la forme suivante :

$$\sum_{j=1}^n w_{i,j} x_j - x_{n+i} + q f_i = -u_i.$$

Dans celles-ci, les sommes $\sum_j w_{i,j}$ sont compensées⁸ par un terme $q f_i$. Si ces sommes sont grandes, alors les valeurs de f_i devront être grandes pour compenser, et Gurobi explorera probablement plus de branches pour les déterminer. Si ces sommes sont plus petites, ce qu'elles deviennent suite au centrage des coefficients $w_{i,j}$ autour de 0, les valeurs des f_i restent alors plus petites : Gurobi peut donc les déterminer plus rapidement.

6.2 Modélisation LLL (erronée)

En section 2, nous avons présenté l'algorithme LLL, en le considérant comme l'état de l'art pour la résolution LWE. Dans une optique d'optimalité, nous avons donc désiré l'utiliser dans notre travail. Nous avons alors commencé à travailler sur une fonction nommée *solve_lwe_lll* a été implémentée. À travers cette dernière, nous utilisons la bibliothèque Python FPyLLL afin de pouvoir exécuter l'algorithme LLL sans avoir à le coder manuellement. Cependant, cela nécessita que nous téléchargions WSL, un sous-système Linux fonctionnant directement à l'intérieur de Windows.

Dans le même esprit que la section précédente (6.1), notre objectif reste de diminuer la somme $\sum_{j=1}^n w_{i,j} x_j$ de nos contraintes. La condition de taille de l'algorithme LLL nous assure justement une amélioration dans cette direction. Nous avons alors tenté d'appliquer LLL directement sur la matrice W de coefficients.

Remarque 2. Rappelons que lorsque nous appliquons LLL sur une matrice, nous obtenons une égalité telle que $W' = UW$, où W' est la matrice post-LLL, où U est la matrice de passage et où W est la matrice initiale. Cette matrice de passage nous est fondamentale car elle nous permet d'ajouter les variables z respectant les contraintes de liaisons $x = Uz$.

8. On peut le voir comme une remise à niveau du modulo.

Des tests pratiques ont été effectués avec $n = 11, t = 2$ et avec un m variable. Pour tout $m < 22$, Gurobi retournait une erreur : les instances étaient insatisfaisables, car les coefficients de la matrice de passage U dépassaient les valeurs numériques maximales prises en charge par Gurobi. En creusant un peu, il était possible de réaliser que comme $W \equiv A_1 A_0^{-1}$, alors W était de taille $(m - n) \times n$. À $m < 2n$, nous obtenions alors une matrice large, pour laquelle il y a plus de colonnes que de lignes : ces colonnes ne peuvent donc pas être indépendantes. Il est impossible d’appliquer LLL sur ce genre d’instance [10], expliquant alors les coefficients dégénérés de U .

À $m \geq 22$, nous obtenons cette fois-ci des solutions. À titre de comparaison, présentons dans le tableau ci-dessous les performances du solveur LWE à centrage modulaire versus le solveur LWE avec LLL sur W . Notons que le solveur LLL utilise aussi le centrage modulaire : il n’est donc pas désavantagé.

TABLEAU 3 – Comparaison du modèle CMOD avec le modèle LLL

Modèle	m	Temps moyen (s)	Temps médian (s)	Statut
CMOD	18	4.3951	0.9208	Résolu
	22	1.6176	1.7051	Résolu
	24	5.8766	1.6375	Résolu
LLL	18	0.3249	0.3249	Non résolu
	22	13.1743	6.3400	Résolu
	24	15.2254	13.7519	Résolu

D’un seul coup d’oeil, il est clair que le solveur LLL est moins efficace que le solveur CMOD, peu importe que le système soit déterminé, sous-déterminé ou surdéterminé. Cet échec s’explique par une utilisation erronée de l’algorithme LLL. Rappelons que ce dernier est un algorithme de réduction de base sur un réseau. Cependant, la matrice W n’est pas représentative du réseau du problème LWE ! Cette dernière n’encode aucunement la structure modulaire du problème. Dans la prochaine section, nous proposons une alternative pour contourner ce problème.

6.3 Modélisation LLL (adéquate)

L'article ressemblant le plus à notre travail actuel est Jiang [3] : il date du 22 juin 2026 (!). Dans celui-ci, le problème LWE est transformé en un problème d'optimisation d'une manière similaire à celle de Shirase. Le partitionnement A_0, A_1 de la matrice A est identique et il y a création d'une matrice $W \equiv A_1 A_0^{-1}$. C'est ensuite que leurs chemins divergent : Shirase pose un problème de PQNE pour un solveur MIP, tandis que Jiang quitte vers un modèle d'optimisation binaire quadratique sans contraintes (QUBO), qu'elle fera résoudre par une Ising Machine, soit un solveur physique s'approchant de l'ordinateur quantique. Cependant, la construction d'une base représentative d'une instance LWE y est exhibée : nous réutilisons alors ce résultat dans notre travail.

Théorème 2. Soit A une matrice aléatoire de taille $m \times n$ initialisant un problème LWE(m, n, q, B) avec $m > n$. Soit A_0 la sous-matrice correspondant aux n premières lignes de A et soit A_1 la matrice correspondant aux $m - n$ lignes restantes. Supposons que A_0 soit inversible, et posons ensuite $W \equiv A_1 A_0^{-1}$. Il est alors possible d'obtenir une base générant le réseau représentatif du problème LWE L_A via la matrice

$$B = \begin{bmatrix} I_{n \times n} & 0_{n \times (m-n)} \\ W_{(m-n) \times n} & q \cdot I_{(m-n) \times (m-n)} \end{bmatrix}.$$

Cherchons à générer cette base depuis le modèle de Shirase (figure 2). Pour commencer, ajoutons les n contraintes triviales suivantes :

$$x_j = x_j \quad \forall j \in \{1, \dots, n\}.$$

Celles-ci correspondent aux composantes de $\mathbf{x}_0$: nous obtenons alors $(m - n) + n = m$ contraintes. En isolant x_{n+i} dans les contraintes de Shirase, nous trouvons :

$$x_{n+i} = \sum_{j=1}^n w_{i,j} x_j + q f_i + u_i, \quad \forall i \in \{1, 2, \dots, m - n\}$$

et en combinant ces équations avec les n contraintes triviales que nous venons de poser, il est possible de trouver le système suivant :

$$\begin{bmatrix} \mathbf{x}_0 \\ \mathbf{x}_1 \end{bmatrix} = \begin{bmatrix} I_{n \times n} & 0_{n \times (m-n)} \\ W_{(m-n) \times n} & q \cdot I_{(m-n) \times (m-n)} \end{bmatrix} \begin{bmatrix} \mathbf{x}_0 \\ \mathbf{f} \end{bmatrix} + \begin{bmatrix} \mathbf{0}_n \\ \mathbf{u} \end{bmatrix}.$$

En posant $(\beta_{i,j}) \in \mathbb{Z}^{m \times m}$ comme l'ensemble des coefficients de la matrice B et en posant $\bar{\mathbf{u}} = [\mathbf{0}_n^\top \mathbf{u}^\top]$, nous trouvons alors le modèle de PQNE suivant, où y_j correspond à la j -ème composante du vecteur $[\mathbf{x}_0^\top, \mathbf{f}^\top]^\top$:

$$\begin{aligned} \text{minimiser : } & \sum_{i=1}^m x_i^2 \\ \text{sujet à : } & \sum_{j=1}^m \beta_{i,j} y_j + \bar{u}_i = x_i, \quad \forall i \in \{1, 2, \dots, m\} \\ & -t \leq x_i \leq t, \quad \forall i \in \{1, 2, \dots, m\} \\ & x_i, y_j \in \mathbb{Z}. \end{aligned}$$

Il est alors possible d'appliquer LLL sur la matrice B . Soit B' la nouvelle matrice et soit β' ses coefficients. Nous obtenons alors le modèle final suivant :

$$\begin{aligned} \text{minimiser : } & \sum_{i=1}^m x_i^2 \\ \text{sujet à : } & \sum_{j=1}^m \beta'_{i,j} y_j + \bar{u}_i = x_i, \quad \forall i \in \{1, 2, \dots, m\} \\ & -t \leq x_i \leq t, \quad \forall i \in \{1, 2, \dots, m\} \\ & x_i, y_j \in \mathbb{Z} \end{aligned}$$

Notons qu'il était nécessaire de remplacer le vecteur $[\mathbf{x}_0^\top, \mathbf{f}^\top]^\top$ par un vecteur y générique compte-tenu du désordre induit par l'application de LLL sur la matrice. Dans le cadre de ce travail, nous laissons Gurobi borner les composantes de ce vecteur y , mais il serait important de la faire manuellement dans un développement futur.

Évaluons maintenant la performance et les limites de ce nouveau modèle, que l'on appellera BLLL. Pour commencer, une série de tests à m variable avec $n = 11, q = 3329$ et $t = 2$ ont été effectués. Nous utilisons les mêmes 30 seeds qu'en figures 6 et 7. Les graphiques suivants présentent nos résultats.

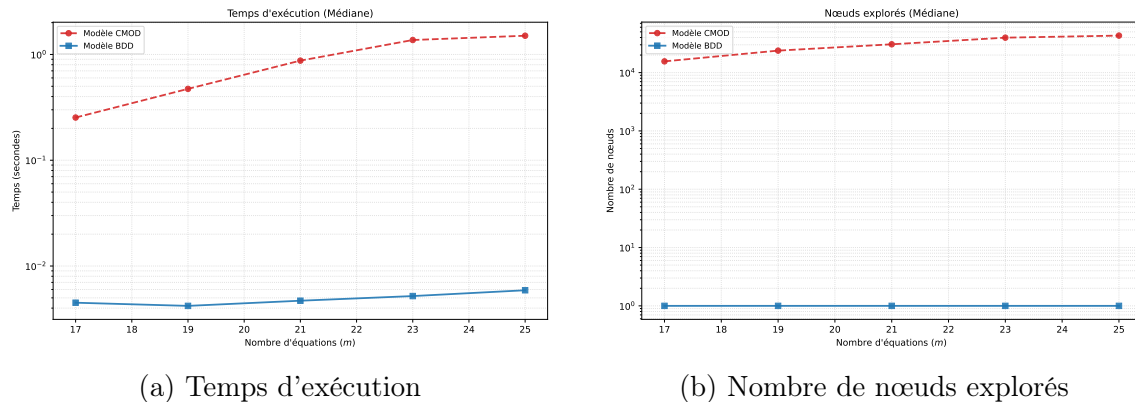


FIGURE 8 – Comparaison (médiane) du modèle CMOD avec le modèle BLLL.

Nous remarquons très rapidement que le modèle BLLL est excessivement plus rapide que notre ancien meilleur modèle, le modèle CMOD. Il n'y a même pas compétition : pour les quadruplets de paramètres que nous avons utilisés, le modèle BLLL arrive à constamment résoudre les instances avec un seul noeud ! Le presolve de Gurobi combiné à la puissance de LLL nous permet de résoudre de petites instances excessivement rapidement.

Par rapport aux anciennes instances limites du modèle de Shirase, l'amélioration du temps de résolution est tout aussi impressionnante. Nous avons fixé ces dernières à $m \approx 35, n \approx 20$, pour un $t \geq 2$ et un $q \geq 101$. Sur de tels problèmes, le modèle initial tombait dans un timeout constant après 15 minutes. Pour des problèmes identiques, notre nouveau solveur prend environ 10^{-2} secondes de moyenne, soit une amélioration d'au moins 4 ordres de grandeur.

Pour conclure, il avait lieu de briser de plus grosses instances afin de déterminer les limites de notre nouveau solveur. Ramenons nous à $q = 3329$ et $t = 2$. Nous travaillons cette fois-ci sur des augmentations progressives et simultanées de n et

m . Par souci de temps et de reproductibilité, nous avons temporairement fixé un timeout à 250 secondes, et nous avons effectué 3 tests (sur les seeds 1,2 et 3). Nous travaillons avec trois progressions différentes : $m = n + 20$, $m = n + 35$ et $m = n + 50$.

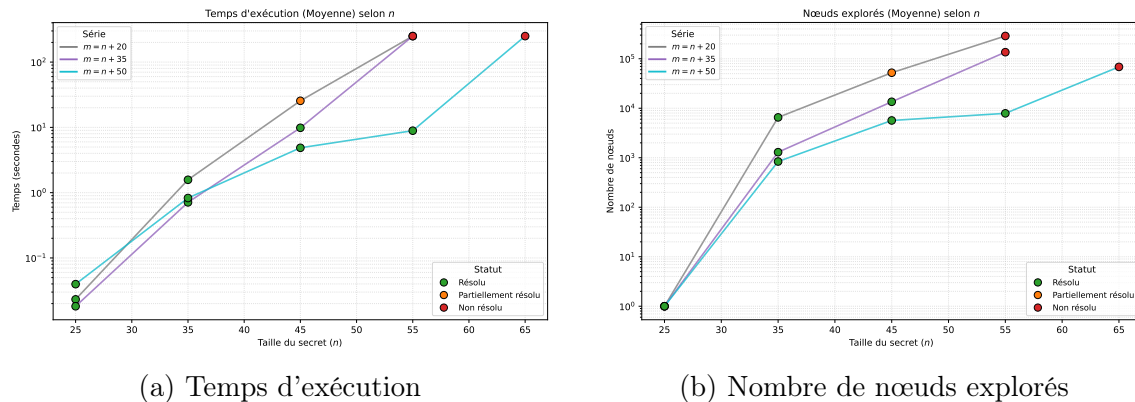


FIGURE 9 – Comparaison (moyenne) de trois séries de couples m, n différentes.

Remarquons que nous sommes capables de briser assez rapidement des instances beaucoup plus volumineuses que celles avec lesquelles nous travaillions plus tôt. Notons qu'aucune des solutions n'a été trouvée en plus de 30 secondes : c'était soit moins, soit un timeout Gurobi. Nous supposons que cela vient d'une faiblesse dans la productivité de Gurobi : lorsque la base que lui donne LLL n'est pas assez orthogonale, il travaille excessivement fort pour retrouver la solution. Souvent, Gurobi semble se «perdre» dans l'espace des solutions. Le nombre d'itérations par noeud de Branch-and-Bound est approximativement deux fois plus élevé lorsque Gurobi ne trouve pas rapidement la solution : le simplexe sous-jacent travaille donc deux fois plus longtemps par noeud d'un problème mal conditionné.

Les tests effectués après $n = 65$ n'ont pas été représentés graphiquement, puisqu'ils furent assez peu concluants, peu importe le terme additif x que nous ajoutons dans $n = m + x$. Le solveur BLLL a réussi une unique fois à casser une instance de taille 125×75 , sur 50 essais. Cette solution fut trouvée en 9 secondes : le système aléatoire initial doit avoir été excessivement bien conditionné par LLL, ce qui retourna une base facile à utiliser pour Gurobi.

Conclusion

Cette étude consistait en une introduction à la cryptographie moderne. En premier lieu, une revue de littérature combinant cryptographie classique et post-quantique a été rédigée. Celle-ci est accessible en [5] : des retours positifs d'étudiants nous ont permis de valider son utilité. Une attention particulière a été accordée aux réseaux euclidiens, et le système LWE, acronyme de *Learning With Errors*, a été le point focal de l'étude.

L'article ayant inspiré ce travail en est un de Shirase [13]. Dans celui-ci, il est exhibé une réduction du problème Search-LWE, soit le problème LWE de base, à un problème de programmation en nombres entiers. À titre de nouveaux développements, le modèle de PQNE correspondant à LWE a été implémenté en Python via le solveur d'optimisation Gurobi. Il a été possible de valider empiriquement la construction mathématique du modèle. Cependant, le solveur tel quel n'était pas très puissant : avec une erreur $t \geq 2$, un module $q \geq 101$ et une matrice de taille $m \times n$ étant de dimensions $\approx 35 \times 20$, le solveur se terminait en un timeout fixé à 15 minutes. Cependant, il est attendu que de telles petites instances de LWE soient résolubles beaucoup plus rapidement.

L'amélioration principale ayant été apportée au modèle est celle d'une simplification de ce dernier utilisant l'algorithme LLL. En construisant notre réseau via une construction similaire à Shirase [13] trouvée en Jiang [3], il était possible d'incorporer une nouvelle base LLL-réduite dans un modèle de PQNE alternatif. Les améliorations apportées par ce changement furent sans équivoque : les anciennes instances limites furent maintenant résolubles en quelques millisecondes. Un mur fut cependant frappé à des instances telles que $n \approx 65$: le solveur tombait encore une fois dans un timeout fixé à 25 minutes. L'instance la plus grande résolue en un temps raisonnable fut une de taille 125×75 , avec $t = 2$ et $q = 3329$. Curieusement, sa solution fut trouvée en moins de 10 secondes. Bien que cela soit une excellente amélioration, ces instances restent d'un ordre de difficulté excessivement bas lorsque comparées aux cryptosystèmes actuels basés sur les réseaux tels que Kyber et Dilithium.

Nous jugeons nos objectifs de l'été accomplis : ce travail pourra définitivement servir de base pour de la recherche future, et aura permis de répondre aux questions ouvertes de Shirase [13]. Ayant eu plus de temps sur l'été, nombreuses auraient été les améliorations ayant pu être apportées aux modèles de PQNE étudiés.

D'un point de vue d'optimisation pure, des techniques classiques viennent à l'esprit et auraient pu être utilisées sur chacun de nos modèles. D'abord, suite à l'implémentation du modulo centré, il aurait été possible de resserrer directement les bornes sur les f_i , réduisant l'espace de recherche. En outre, la Pr. Yu proposa l'ajout manuel de coupes pouvant affiner l'espace de recherche de Gurobi, possiblement à l'aide d'inégalités valides fortes. Ces contraintes permettraient potentiellement de rapprocher l'espace de recherche à celui de l'espace de l'enveloppe convexe du problème.

Sur le modèle final, il aurait été intéressant de troquer l'algorithme LLL pour l'algorithme BKZ, le standard d'industrie actuel (BKZ étant un LLL exécuté par blocs). La base que retourne BKZ propose une garantie d'orthogonalité et de vecteurs de petites tailles plus forte que LLL : il est naturel de supposer que l'espace engendré par une réduction BKZ aurait permis un travail beaucoup plus efficace de Gurobi.

Pour terminer, il serait aussi intéressant d'analyser plus en détail la résolution d'instance. Il a été possible de remarquer qu'avec le modèle optimisé, il n'a jamais fallu plus de 25 secondes à Gurobi pour déterminer une solution fonctionnelle. Le solveur fonctionnait donc un peu sous un principe de tout ou rien : soit la résolution échoue par manque de temps à 25 minutes, soit la solution est découverte excessivement rapidement. Il pourrait alors être intéressant de caractériser les instances à succès et d'observer leurs différences par rapport aux instances à échecs. Cela viendrait potentiellement expliquer le surprenant succès à 125×75 .

Références

- [1] Avanzi, R., Bos, J., Ducas, L., Kiltz, E., Lepoint, T., Lyubashevsky, V., Schanck, J. M., Schwabe, P., Seiler, G., Stehlé, D., et al. (2019). Crystals-kyber algorithm specifications and supporting documentation. *NIST PQC Round*, 2(4) :1–43.
- [2] Conforti, M., Cornuéjols, G., and Zambelli, G. (2014). *Getting Started*, pages 1–44. Springer International Publishing, Cham.
- [3] Jiang, S. (2026). When the learning with errors problem meets the coherent ising machine : A penalty-free algorithm-hardware co-design. *arXiv preprint arXiv :2606.22843*.
- [4] Lenstra, A. K., Lenstra, H. W., and Lovász, L. (1982). Factoring polynomials with rational coefficients.
- [5] Lessnick, A. (2026a). Application de la programmation en nombres entiers aux cryptosystèmes post-quantiques basés sur les réseaux. Rapport de recherche (IFT3150), Département d’informatique et de recherche opérationnelle, Université de Montréal.
- [6] Lessnick, A. (2026b). Implémentation de la réduction lwe et résolution via programmation en nombres entiers avec gurobi. https://github.com/alexlessn/solveur_lwe. Code source Python. Consulté le 10 juillet 2026.
- [7] Lv, L., Yan, B., Wang, H., Ma, Z., Fei, Y., Meng, X., and Duan, Q. (2022). Using variational quantum algorithm to solve the LWE problem. *Entropy*, 24(10) :1428.
- [8] Menezes, A. (2026). A gentle introduction to lattice-based cryptography. *Cryptography ePrint Archive*.
- [9] National Institute of Standards and Technology (2024). Module-lattice-based key-encapsulation mechanism standard. Federal Information Processing Standards Publication (FIPS) NIST FIPS 203, U.S. Department of Commerce.
- [10] Nguyen, P. Q. and Vallée, B. (2010). *The LLL algorithm*. Springer.

- [11] Regev, O. (2009). On lattices, learning with errors, random linear codes, and cryptography. *Journal of the ACM (JACM)*, 56(6) :1–40.
- [12] Regev, O. (2010). The learning with errors problem. *Invited survey in CCC*, 7(30) :11.
- [13] Shirase, M. (2023). Reduction of search-lwe problem to integer programming problem. *Cryptology ePrint Archive*.
- [14] Shor, P. W. (1999). Polynomial-time algorithms for prime factorization and discrete logarithms on a quantum computer. *SIAM Review*, 41(2) :303–332.
- [15] Silverman, J. H., Pipher, J., and Hoffstein, J. (2008). *An introduction to mathematical cryptography*, volume 1. Springer.
- [16] TU Darmstadt (2026). Lattice challenge – lwe challenge. https://www.latticechallenge.org/lwe_challenge/challenge.php. Consulté le 5 août 2026.

Remerciements

J'aimerais débiter en remerciant ma superviseuse Kimberly Yu pour son mentorat et sa confiance. Ce fut ma première exploration dans le monde de la recherche : elle a su m'y guider avec brio. J'aimerais aussi remercier Zachary et Théodore qui me permirent de décompresser lorsque mon cerveau devenait mathématiquement surchargé. Ce fut un deux mois assez intense ! Pour finir, merci à Stella pour son soutien et sa compréhension. Ce sont nos sorties en soirée qui me motivent à travailler en matinée.

Annexe

A Implémentation de la réduction LWE

```
def solve_lwe(A, b, q, t, time_limit=1800):
    m, n = A.shape
    m_n = m - n
    A_0, A_1 = A[0:n, :], A[n:m, :]
    b_0, b_1 = b[:n], b[n:]
    A_0inv = np.array(Matrix(A_0).inv_mod(q)).astype(int)
    W = (A_1 @ A_0inv) % q
    u = (b_1 - W @ b_0) % q

    modele = gp.Model("SolveurLWE")
    modele.setParam('TimeLimit', time_limit)
    modele.setParam('SolutionLimit', 1)

    x = modele.addVars(m, vtype=GRB.INTEGER, lb=-t, ub=t, name="x")
    f_inf = -1 * ((t * (n * q - n + 1) + q - 1) // q)
    f_sup = (t * (n * q - n + 1)) // q
    f = modele.addVars(m_n, vtype=GRB.INTEGER, lb=f_inf, ub=f_sup, name="f")

    for i in range(m_n):
        cote_gauche = gp.quicksum(W[i, j] * x[j] for j in range(n)) - x[n+i] + q *
        f[i]
        modele.addConstr(cote_gauche == -u[i], name=f"eq_{i}")

    modele.setObjective(gp.quicksum(x[k] * x[k] for k in range(m)), GRB.MINIMIZE)
    modele.optimize()

    nodes, runtime = modele.NodeCount, modele.Runtime
    if modele.status in [GRB.OPTIMAL, GRB.TIME_LIMIT, GRB.SOLUTION_LIMIT] and
    modele.SolCount > 0:
        e_sol = np.array([int(x[k].X) for k in range(m)])
        s_hat = (A_0inv @ (b_0 - e_sol[:n])) % q
        return s_hat, e_sol, nodes, runtime

    return None, None, nodes, runtime
```